Diviser pour régner 2

Dichotomie & rotation

Vocabulaire :

- Diviser pour régner / recherche dichotomique / terminaison et correction d'un algorithme

Prérequis :

- La notion de récursivité doit avoir été vue.
- Avoir fait la première séance sur diviser pour régner.
- Connaître et comprendre la méthode diviser pour régner.

Objectifs :

- Écrire un algorithme utilisant la méthode « diviser pour régner »
- Comprendre que la recherche dichotomique est un exemple de la méthode diviser pour régner d'un algorithme récursif.
- Faire la rotation d'une image bitmap d'un quart de tour avec un coût en mémoire constant.

Dans cette séance nous allons travailler sur deux exemples utilisant la méthode diviser pour régner.

Le premier exemple a été vu l'an dernier en classe de première, il s'agit de la recherche dichotomique. Pas de panique si tu l'as oubliée, on revoit ça en douceur. C'est une bonne occasion d'utiliser la récursivité avec la technique : j'écris au début de l'algorithme la condition pour sortir de la récursivité, puis je code le passage contenant les appels récursifs qui doivent mener à vers la condition de sortie.

La seconde est la rotation d'une image d'un quart de tour. C'est l'algorithme utilisé par ton smartphone pour passer de l'affichage portrait à paysage.

I- LA RECHERCHE DICHOTOMIQUE

A- Principe de la méthode Diviser pour régner

- Un algorithme de type diviser pour régner se décompose en trois parties :
  1. **Diviser** : On divise le problème initial en plusieurs sous-problèmes plus petits que le problème initial.
  2. **Régner :** il s'agit de la résolution récursive des sous-problème : On résout chacun des sous-problèmes. La méthode est particulièrement efficace si on peut le faire de manière récursive. Il faut pour cela que les sous-problèmes soient des instances du problème initial avec de plus petits paramètres.
  3. **Combiner** : On peut reconstituer la solution du problème initial avec les solutions des différents sous-problèmes.

  Condition sur l'appel récursif. L'étape cruciale de cette méthode de recherche est l'étape de division. Pour être efficace, cette stratégie doit diviser le problème. Pour être plus précis, si on applique un algorithme de type diviser pour régner à une donnée de taille N, alors les appels récursifs doivent se faire sur des données de tailles N/k, avec k un entier strictement plus grand que 1.

B- Le principe de la recherche dichotomique

- On se donne une liste triée, et on veut pouvoir déterminer si un élément se trouve ou non dans la liste. On peut spécifier le problème de la manière suivante :
  - Données : Une liste `lst` d'éléments, et un élément `elt`.
  - Précondition : La liste `lst` doit être triée selon l'ordre croissant. Autrement dit, on doit avoir `lst[i] <= lst[j]` pour tout couple valide d'indices `i` et `j`, tel que `i < j`. L'élément `elt` doit être comparable avec tous les éléments de `lst`.
  - Résultat : `True` si `elt` est un élément de `lst`, et `False` sinon.
  - Postcondition : Aucunes.

C- Implémenter la recherche dichotomique en Python

- Utilisons la méthode **Diviser pour régner**. On commence notre recherche par le milieu de la liste. Si l'élément central de `lst` est `elt`, alors on renvoie `True`. Sinon, on peut utiliser le fait que la liste est ordonnée pour éliminer une moitié des éléments de `lst`. On applique ensuite récursivement la recherche dichotomique sur la moitié restante des éléments.
- Le cas minimum assurant l'arrêt des appels récursif est le cas de la liste vide : si la liste `lst` est vide, elle ne contient pas l'élément `elt`, et on renvoie `False`.

Exercice 1 :

1) Écris le pseudocode de l'algorithme de recherche dichotomique en version récursive bien entendu.
2) Compare, sans trop de bruit, ton pseudocode avec ton voisin, puis fais-le vérifier par ton professeur.
3) Écris une fonction `rechercheDichoRec(lst,elt)`.
4) Crée une liste `lst_croissante` par compréhension avec des valeurs croissantes à l'intérieur.
5) Teste ta recherche dichotomique on demandant à l'utilisateur de rentrer l'élément à tester.

[Ecris directement ta réponse ici sauvegarde ensuite ton fichier pdf.]

D- Terminaison et correction de l'algorithme

IL est important, surtout quand il est récursif, de vérifier qu'un algorithme se termine. On cherche alors la preuve de la terminaison de l'algorithme.
D'autre part, on doit vérifier que l'algorithme donne le bon résultat. On appelle cela la preuve de la correction de l'algorithme.
Les voici à titre d'exemple pour la recherche dichotomique.

Preuve de terminaison. L'appel récursif se fait sur une liste de taille strictement plus petite que la taille de la liste passée en entrée. La condition d'arrêt portant sur la taille de la liste (on vérifie si la liste est vide), les appels récursifs vont se terminer.
Preuve de correction. Si la liste est vide, notre algorithme renvoie clairement `False`. De même, lors du premier test, notre algorithme vérifie si l'élément d'indice `k` est égale à `elt`. Si c'est le cas, on renvoie `True`.
Si `lst[k]` n'est pas égal à `lst`, on teste si `elt < lst[k]`. Si c'est le cas, alors il est inutile de tester les éléments d'indices plus grands que `k`. En effet, si i est un indice valide de `lst`, et que `k < i`, alors on a `elt < lst[k] <= lst[i]`, puisque les éléments de la liste sont rangés en ordre croissant.
Nous ne devons alors que tester les éléments d'indice `i < k`. C'est ce que fait l'appel récursif.
Un raisonnement tout à fait symétrique s'applique si `elt > lst[k]`.

E- Complexité temporelle de l'algorithme

- La **complexité** de cette algorithme est à connaître, il est en **O(log$_2$(n))**.

- Voici la démonstration mathématique pour information. Elle n'est pas à connaître.
  Nous allons évaluer la complexité de cette fonction par le nombre de comparaisons qu'elle devra faire entre `elt` est les éléments de la liste `lst` pour se terminer. On notera C la fonction de complexité, et C(n) le nombre de comparaisons nécessaires à la fonction `rechercheDichoRec` pour terminer sur une liste de n éléments. On peut déjà noter que :
  - Si la liste lst est vide, la fonction `rechercheDichoRec` n'exécute aucune comparaison. On a donc C(0) = 0.
  - Si la liste `lst` n'est pas vide, le fonction `rechercheDichoRec` fait une seule comparaison, puis un appel récursif à la fonction sur une liste de taille n/2 dans le pire des cas. On a donc la relation de récurrence suivante: C(n) = C(n/2)+1 , dans le pire des cas.

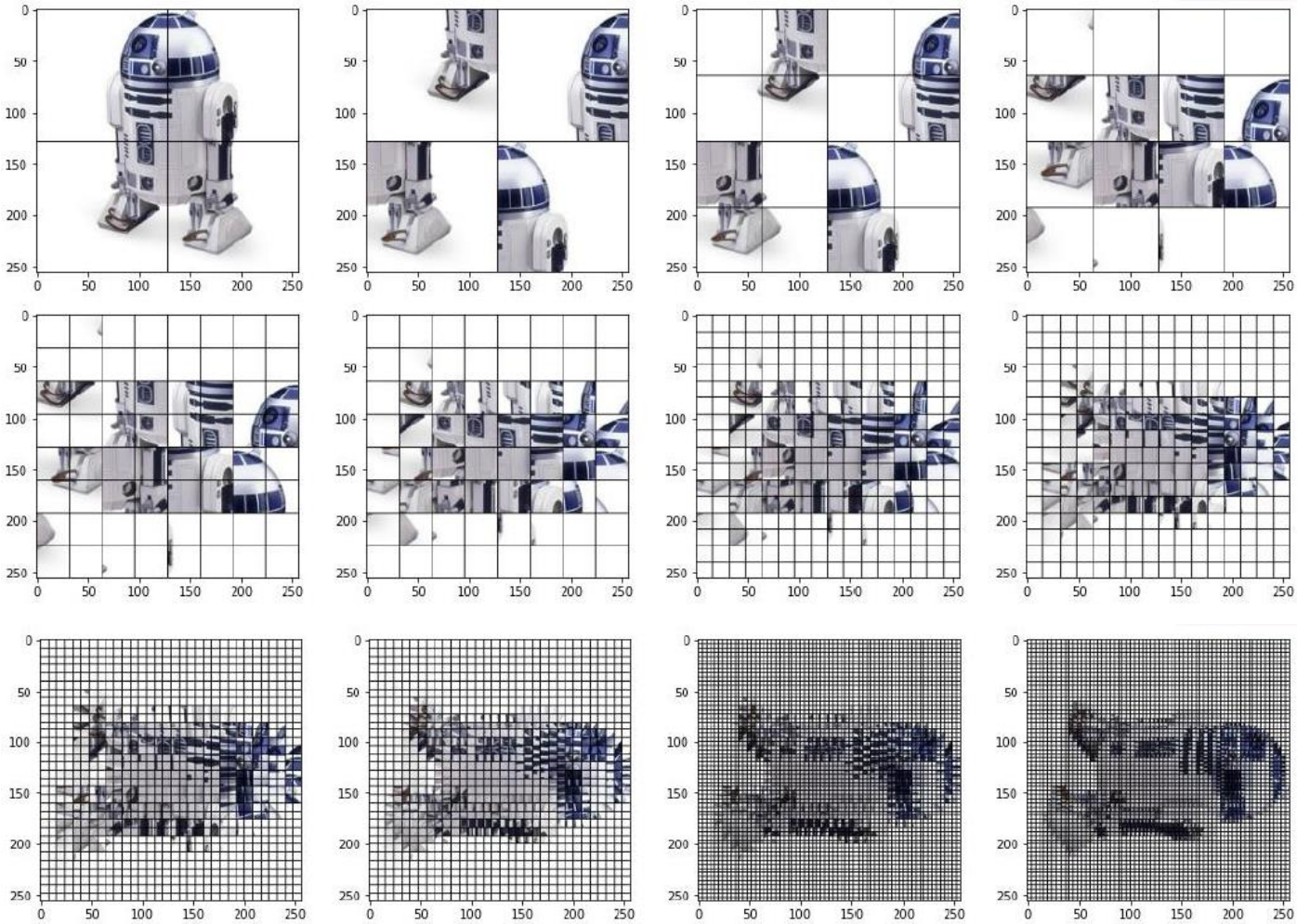
  Il nous faut donc résoudre le système suivant : $C(n) = \begin{cases} 0 & \text{si } n = 0 \\ C\left(\frac{n}{2}\right) + 1 & \text{si } n > 0 \end{cases}$

  On commence par un cas particulier : si n est une puissance de 2, c'est à dire si n est de la forme n = 2^k, avec k un entier naturel. Dans ce cas, on peut résoudre le système par "extension" de la formule : C(n) = C(2^k) = C(2^{k-1})+1 = C(2^{k-2})+2 = C(0)+k
  Dans le cas de n = 2^k, on a donc C(n) = k = log$_2$(n).
  On peut maintenant généraliser pour d'autres valeurs de n. Si on a une liste de n éléments, on peut toujours encadrer n par deux puissances de 2 : on pourra toujours trouver un nombre entier naturel k tel que $2^k < n < 2^{k+1}$.
  Par croissance de la fonction de complexité, on en déduit :
  C(2^k) ≤ C(n) ≤ C(2^{k+1}) donc k ≤ C(n) ≤ k+1
  Or on a k = log$_2$(n). On en déduit l'inégalité : C(n) ≤ log$_2$(n) +1 ≤ log$_2$(n) +1
  La fonction de complexité est donc de classe O(log$_2$(n)).

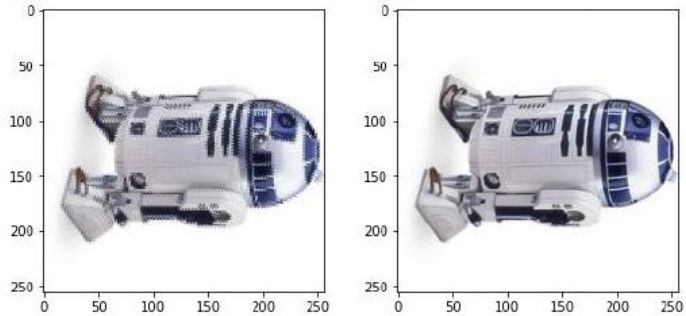
II- LA ROTATION D'UNE IMAGE D'UN QUART DE TOUR

A- Principe de l'algorithme

- La méthode utilisée est valable sur les images carrées dont le côté est une puissance de 2.
- Pour effectuer une rotation d'un quart de tour, on peut diviser l'image en 4, effectuer une permutation circulaire des quatre quadrants, puis appliquer récursivement le même processus à chaque quadrant, jusqu'à obtenir des quadrants contenant un seul pixel.



On poursuit sans représenter le quadrillage devenu trop fin pour les deux dernières étapes.



B- Implémentation en Python de l'algorithme

Le code suivant applique ce procédé, mais pas dans le même ordre que sur les images précédentes…

```
import matplotlib.pyplot as plt
from PIL import Image
import numpy as np
def rotate(image):
    n = len(image)
    if n > 1:
        temp = np.copy(image[n//2:, :n//2])
        image[n//2:, :n//2] = rotate(image[n//2:, n//2:])
        image[n//2:, n//2:] = rotate(image[:n//2, n//2:])
        image[:n//2, n//2:] = rotate(image[:n//2, :n//2])
        image[:n//2, :n//2] = rotate(temp)
    return image

image = np.array(Image.open('animal.jpg'))
fig = plt.figure("image de départ")
plt.imshow(image)
plt.show()

image = rotate(image)

fig = plt.figure("image finale")
plt.imshow(image)
plt.show())
```

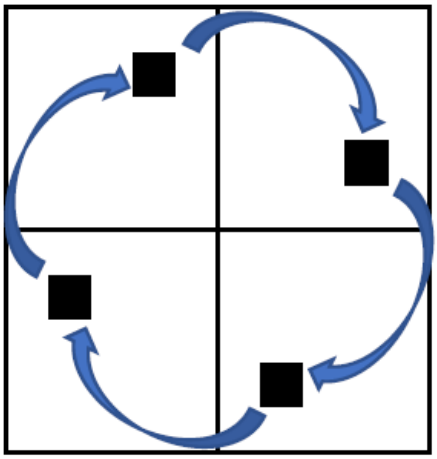
Exercice 2 :

1) Quel est le cas qui permet de stopper la récursivité ?
2) Copie-colle ce code sur un fichier que tu nommeras `rotation1.py`
3) Colle dans le même dossier le fichier nécessaire pour l'exécution de ce programme.
4) Lance le programme. L'image originale s'ouvre. Ferme l'image. Après le déroulement de la fonction rotation, une seconde image s'ouvre. Quelle est la transformation réalisée ?
5) De quoi se nourrit habituellement ce genre d'animal ?

- Cet algorithme est très joli, mais peu efficace.
  Sur l'exemple de taille 256x256, chaque pixel est déplacé 8 fois, alors qu'un seul déplacement est suffisant.

C- Un algorithme plus efficace

- Il s'agit cette fois de faire la rotation avec une méthode valable sur toutes les images carrées mais où lors du déplacement de chaque pixel dans le quadrant suivant, on peut le placer directement à la bonne position et éviter les appels récursifs de la méthode précédente.



Exercice 3 :

1) Crée un nouveau fichier intitulé `rotation2.py`.
2) Recopie et complète le code ci-dessous pour appliquer une rotation d'un quart de tour à l'image représentée par le tableau de nombres image.

```
def rotate(image):
    n = len(image)
    for i in range(n//2):
        for j in range(n//2):
            # partie à compléter pour déplacer
            # les pixels des quatre quadrants
    return image
```

Aide : il s'agit pour cela de commencer par remplacer les coordonnées d'un pixel (par exemple celui du quadrant en haut à gauche par celle du pixel en bas à gauche). Ensuite on applique de nouveau la méthode diviser pour régner.

3) Applique deux fois la rotation sur le fichier `tete.jpg`